Analyses: Classification Performance

3/1/2024

Description

This file enables post-hoc analyses of the performance values (F1-scores, accuracies, p-values) of the KNN and MLP classifications. It provides density distributions for these performance metrics.

Note: The metrics are derived by using the confusionMatrix() function of the caret package. See the code files in analyses_Classification_KNN.Rmd and analyses_Classification_MLP.Rmd for the actual implementation. In the case of a two class classification problem (e.g. Aurignacien vs. Uruk V), a confusion matrix is given as

```
knitr::include_graphics("~/Github/UpperPalCombinatorics/figures_external/confusionMatrix.png")
```

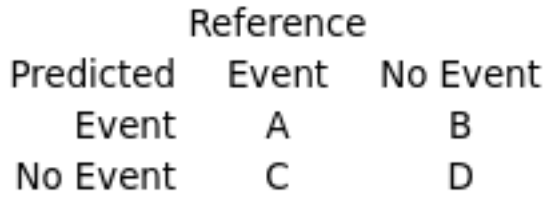


Figure 1: Confusion matrix as given in the confusionMatrix() documentation

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: install.packages("ggplot2").

```
library(ggplot2)
library(ggExtra)
library(gridExtra)
library(ggpubr)
library(ggridges)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v lubridate 1.9.3     v tibble    3.2.1
## v purrr     1.0.2     v tidyr     1.3.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::combine() masks gridExtra::combine()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(data.table)
```

```
##
## Attaching package: 'data.table'
##
## The following objects are masked from 'package:lubridate':
##
##     hour, isoweek, mday, minute, month, quarter, second, wday, week,
##     yday, year
##
## The following objects are masked from 'package:dplyr':
##
##     between, first, last
##
## The following object is masked from 'package:purrr':
##
##     transpose
```

Multi-layer perceptron (MLP)

Load Data

```
# list files in the performance folders
files.mlp.performance <- list.files(path = "~/Github/UpperPalCombinatorics/results/classifier/MLP/perfo
```

Preprocessing

```
# initialize empty data frame
performance.mlp.combined <- data.frame(hidden.structure = character(0),
                                       hidden.size = numeric(0),
                                       hidden.depth = numeric(0),
                                       err.fct = character(0),
                                       act.fct.name = character(0),
                                       algorithm = character(0),
                                       baseline = numeric(0),
                                       accuracy = numeric(0),
                                       accuracy.pvalue = numeric(0),
                                       precision = numeric(0),
                                       recall = numeric(0),
                                       f1 = numeric(0),
                                       accuracy.pvalue.corrected = numeric(0),
                                       comparison = character(0))

# loop through all files and combine them
for (file in files.mlp.performance){
  performance.data <- read.csv(file)
  # apply bonferroni correction
  performance.data$accuracy.pvalue.corrected <- p.adjust(performance.data$accuracy.pvalue,
                                 method = "bonferroni",
                                 n = nrow(performance.data))
```

```
  # get the subcorpora involved in the binary classification
  filename.parts <- unlist(strsplit(basename(file), "\\_|\\."))
  subcorpus1 <- filename.parts[3]
  subcorpus2 <- filename.parts[4]
  performance.data$comparison <- rep(paste(subcorpus1, "vs.", subcorpus2),
                                     nrow(performance.data))
  # add to data frame
  performance.mlp.combined <- rbind(performance.mlp.combined, performance.data)
}
head(performance.mlp.combined)
```

```
##   hidden.structure hidden.size hidden.depth err.fct act.fct.name algorithm
## 1            3,3,2           8            3     sse         tanh    rprop+
## 2            3,1,1           5            3     sse         tanh    rprop+
## 3                3           3            1     sse         tanh    rprop+
## 4            5,3,1           9            3     sse         tanh    rprop+
## 5              2,1           3            2     sse         tanh    rprop+
## 6          2,2,1,2           7            4     sse         tanh    rprop+
##   baseline accuracy accuracy.pvalue precision recall   f1 test.n
## 1     0.83     0.89    0.0006721195      0.87   0.43 0.58    350
## 2     0.83     0.89    0.0006721195      0.71   0.62 0.66    350
## 3     0.83     0.90    0.0002092660      0.75   0.60 0.67    350
## 4     0.83     0.87    0.0351126351      0.61   0.58 0.60    350
## 5     0.83     0.90    0.0002092660      0.76   0.58 0.66    350
## 6     0.83     0.87    0.0172588051      0.68   0.47 0.55    350
##   accuracy.pvalue.corrected                          comparison
## 1                0.04704836 Cuneiform (Uruk III) vs. Aurignacien
## 2                0.04704836 Cuneiform (Uruk III) vs. Aurignacien
## 3                0.01464862 Cuneiform (Uruk III) vs. Aurignacien
## 4                1.00000000 Cuneiform (Uruk III) vs. Aurignacien
## 5                0.01464862 Cuneiform (Uruk III) vs. Aurignacien
## 6                1.00000000 Cuneiform (Uruk III) vs. Aurignacien
```

Select comparisons to further analyse.

```
# select only comparisons involving the Aurignacien
performance.mlp.combined <- performance.mlp.combined[performance.mlp.combined$comparison %like% "Aurigna
```

Density Plots

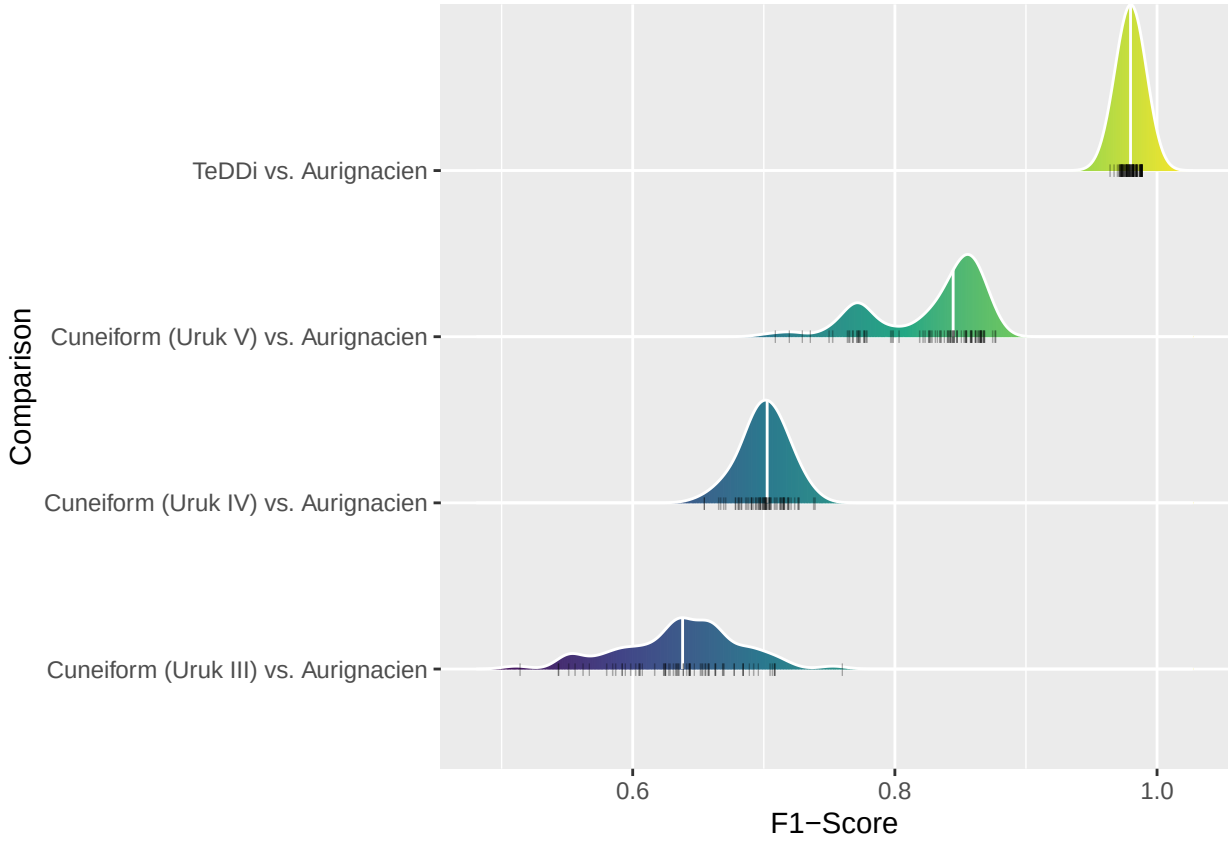
F1 Distributions

Distributions of F1-scores by comparison.

```
f1.plot <- ggplot(performance.mlp.combined, aes(x = jitter(f1, amount = 0),
                                                y = fct_reorder(comparison, f1),
                                                fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                               quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = "F1-Score", option = "D") +
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +
  xlab("F1-Score") +
  ylab("Comparison") +
```

```
  theme(legend.position = "none")
f1.plot
```

```
## Picking joint bandwidth of 0.01
```

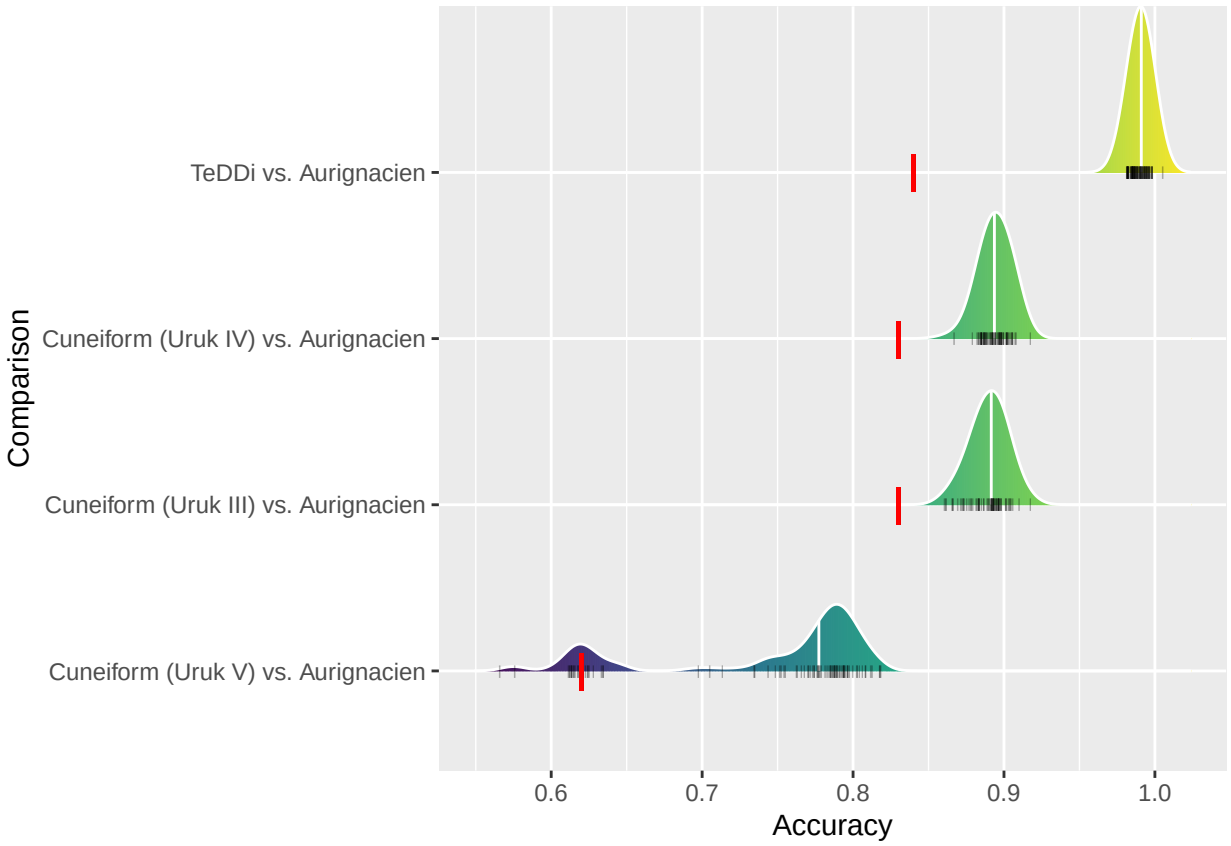


Accuracy Distributions

Distributions of accuracies by comparison.

```
accuracy.plot <- ggplot(performance.mlp.combined, aes(x = jitter(accuracy, amount = 0),
                                                      y = fct_reorder(comparison, accuracy),
                                                      fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                               quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = "Accuracy", option = "D") +
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +
  geom_point(shape = 124, size = 5, aes(x = baseline, y = comparison), color = "red") +
  xlab("Accuracy") +
  ylab("Comparison") +
  theme(legend.position = "none")
accuracy.plot
```

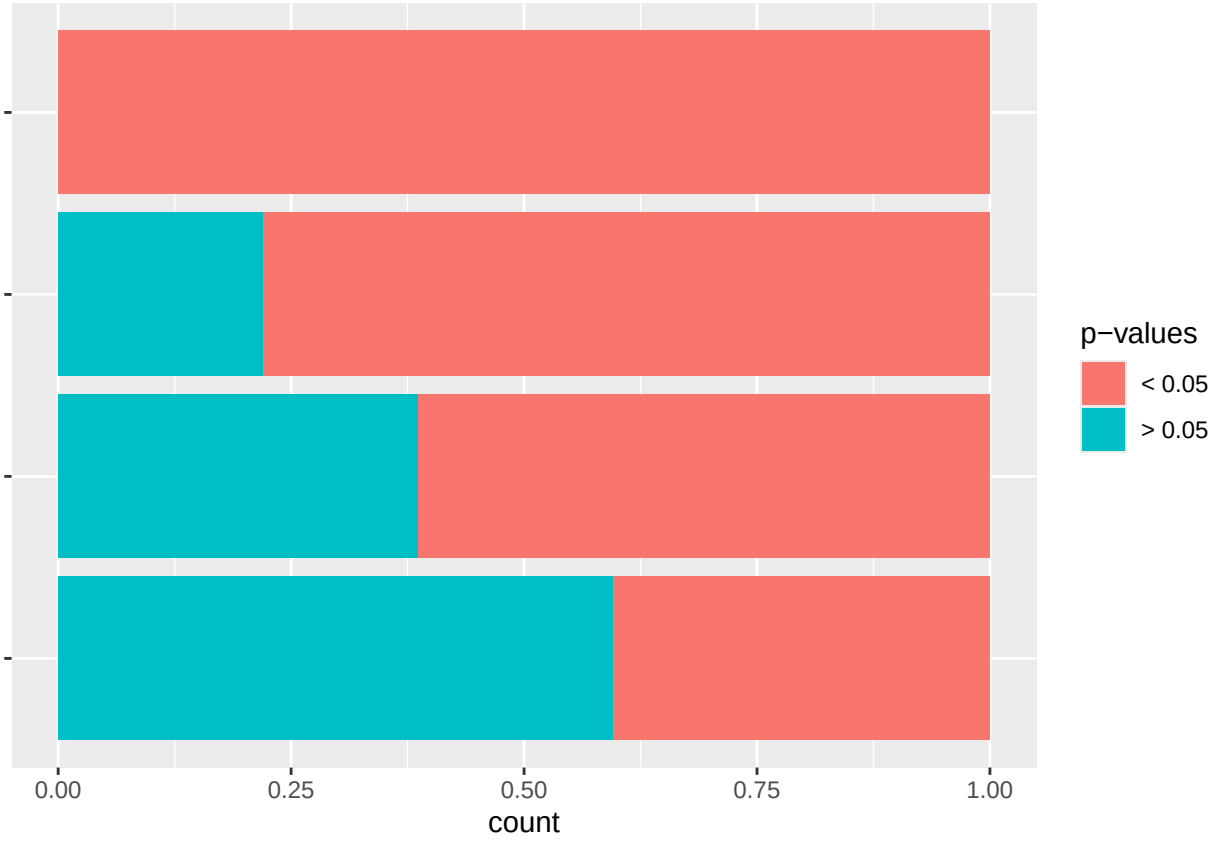
```
## Picking joint bandwidth of 0.00831
```



P-value Distributions

Distributions of p-values by comparison.

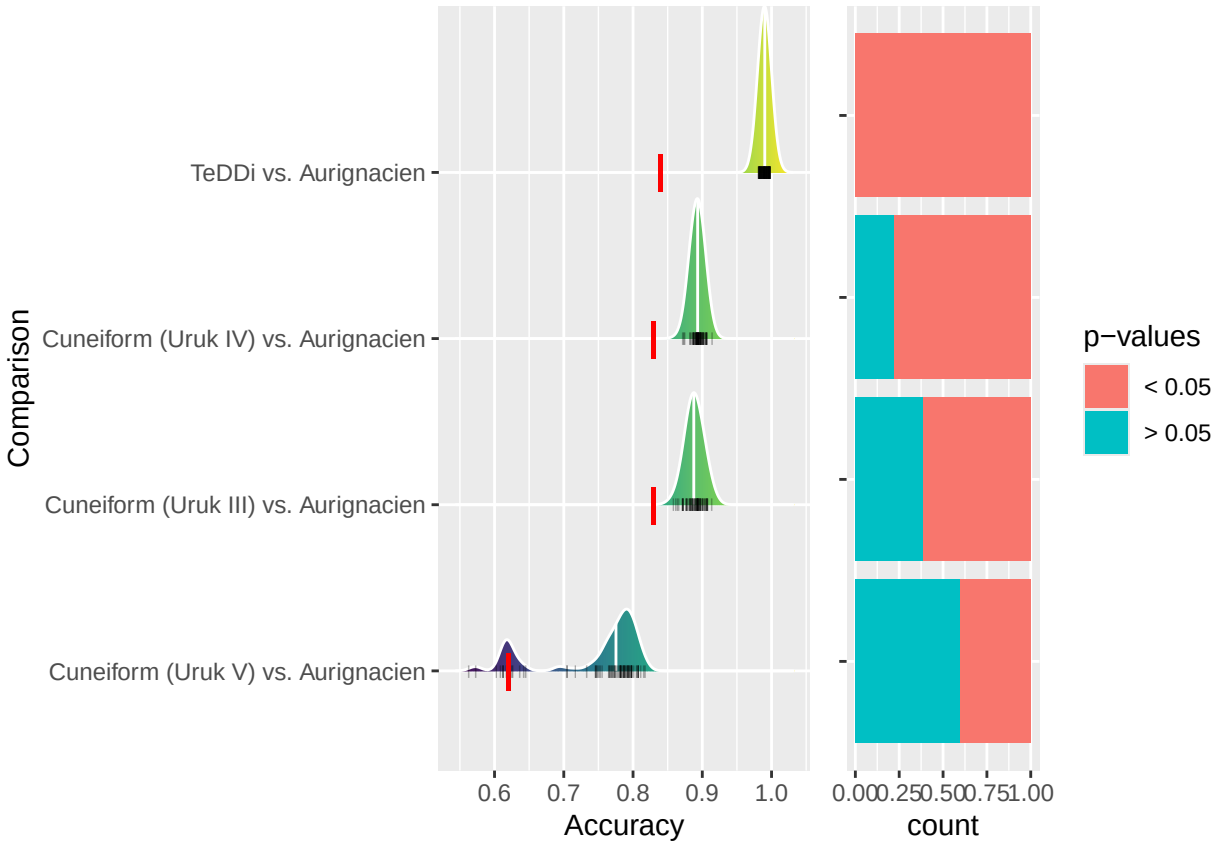
```
# get split between cases where p < 0.05 and p > 0.05
performance.mlp.combined$accuracy.pvalue.corrected <-
  ifelse(performance.mlp.combined$accuracy.pvalue.corrected < 0.05, "< 0.05", "> 0.05")

# create plot
pvalue.plot <- ggplot(performance.mlp.combined, aes(y = fct_reorder(comparison, accuracy),
                                                    fill = as.factor(accuracy.pvalue.corrected))) +
  geom_bar(stat = "count", position = "fill") +
  theme(axis.title.y = element_blank(),
        axis.text.y = element_blank()) +
  scale_fill_discrete(name = "p-values")
pvalue.plot
```



Combine Plots

```
combined.plot <- grid.arrange(accuracy.plot, pvalue.plot, ncol = 2, widths = c(2, 1))
```

```
## Picking joint bandwidth of 0.00897
```



Safe to file.

```
ggsave("~/Github/UpperPalCombinatorics/figures/performancePlot_MLP.pdf",
       combined.plot, dpi = 300, width = 10, height = 2, device = cairo_pdf)
```

K-Nearest-Neighbors (KNN)

Load Data

```
# list files in the performance folders
files.knn.performance <- list.files(path = "~/Github/UpperPalCombinatorics/results/classifier/KNN/perfo
```

Preprocessing

```
# initialize empty data frame
performance.knn.combined <- data.frame(k = numeric(0),
                                       baseline = numeric(0),
                                       accuracy = numeric(0),
                                       accuracy.pvalue = numeric(0),
                                       precision = numeric(0),
                                       recall = numeric(0),
                                       f1 = numeric(0),
                                       accuracy.pvalue.corrected = numeric(0),
```

```
                                    comparison = character(0))

# loop through all files and combine them
for (file in files.knn.performance){
  performance.data <- read.csv(file)
  # apply bonferroni correction
  performance.data$accuracy.pvalue.corrected <- p.adjust(performance.data$accuracy.pvalue,
                                method = "bonferroni",
                                n = nrow(performance.data))
  # get the subcorpora involved in the binary classification
  filename.parts <- unlist(strsplit(basename(file), "\\_|\\."))
  subcorpus1 <- filename.parts[3]
  subcorpus2 <- filename.parts[4]
  performance.data$comparison <- rep(paste(subcorpus1, "vs.", subcorpus2),
                                     nrow(performance.data))
  # add to data frame
  performance.knn.combined <- rbind(performance.knn.combined, performance.data)
}
head(performance.knn.combined)
```

```
##   k baseline accuracy accuracy.pvalue precision recall   f1 test.n
## 1 1     0.83     0.88    0.0077364322      0.68   0.53 0.60    350
## 2 2     0.83     0.87    0.0116927190      0.67   0.52 0.58    350
## 3 3     0.83     0.88    0.0031464871      0.72   0.52 0.60    350
## 4 4     0.83     0.89    0.0019315582      0.74   0.52 0.61    350
## 5 5     0.83     0.90    0.0002092660      0.79   0.55 0.65    350
## 6 6     0.83     0.89    0.0006721195      0.76   0.53 0.63    350
##   accuracy.pvalue.corrected                          comparison
## 1               0.077364322 Aurignacien vs. Cuneiform (Uruk III)
## 2               0.116927190 Aurignacien vs. Cuneiform (Uruk III)
## 3               0.031464871 Aurignacien vs. Cuneiform (Uruk III)
## 4               0.019315582 Aurignacien vs. Cuneiform (Uruk III)
## 5               0.002092660 Aurignacien vs. Cuneiform (Uruk III)
## 6               0.006721195 Aurignacien vs. Cuneiform (Uruk III)
```

Select comparisons to further analyse.

```
# select only comparisons involving the Aurignacien
performance.knn.combined <- performance.knn.combined[performance.knn.combined$comparison %like% "Aurigna
```

Density Plots

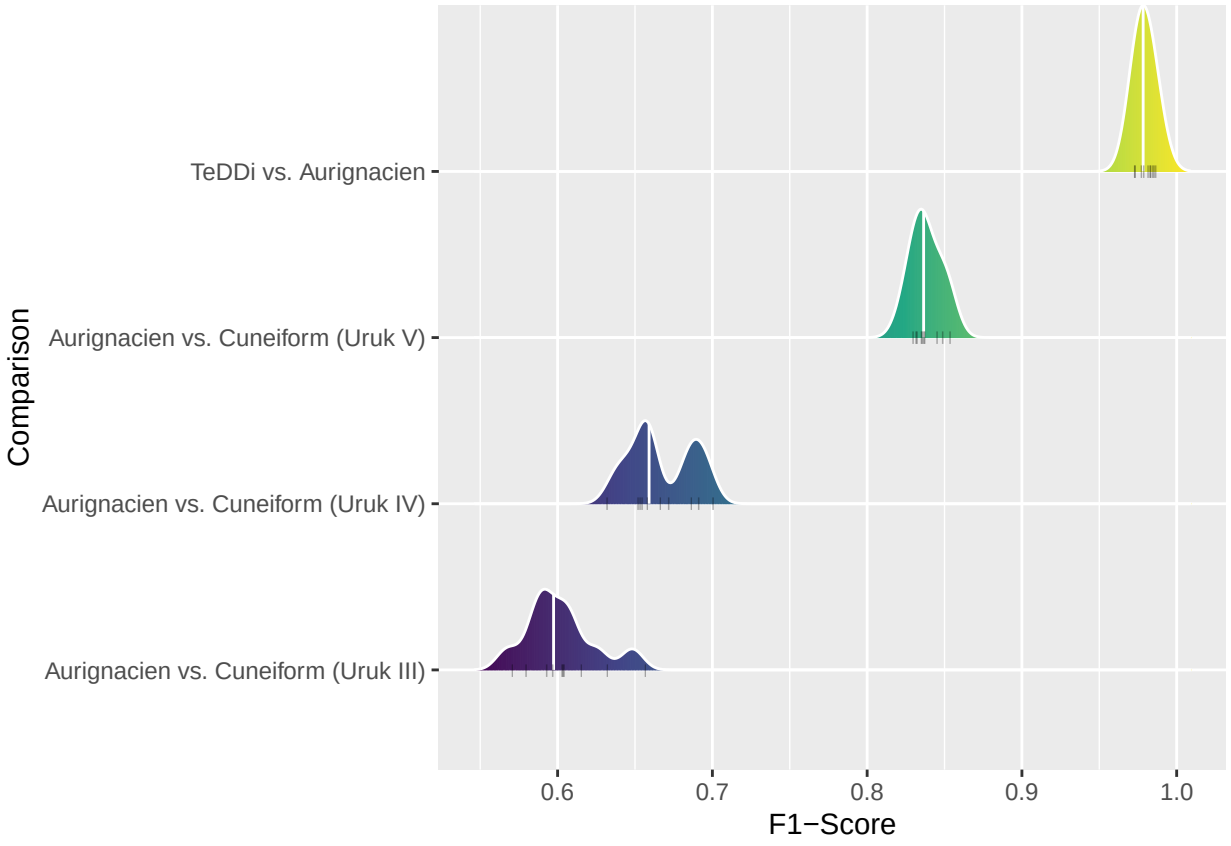
F1 Distributions

Distributions of F1-scores by comparison.

```
f1.plot <- ggplot(performance.knn.combined, aes(x = jitter(f1, amount = 0),
                                                y = fct_reorder(comparison, f1),
                                                fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                               quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = "F1-Score", option = "D") +
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +
  xlab("F1-Score") +
```

```
  ylab("Comparison") +
  theme(legend.position = "none")
f1.plot
```

```
## Picking joint bandwidth of 0.00708
```



Accuracy Distributions

Distributions of accuracies by comparison.

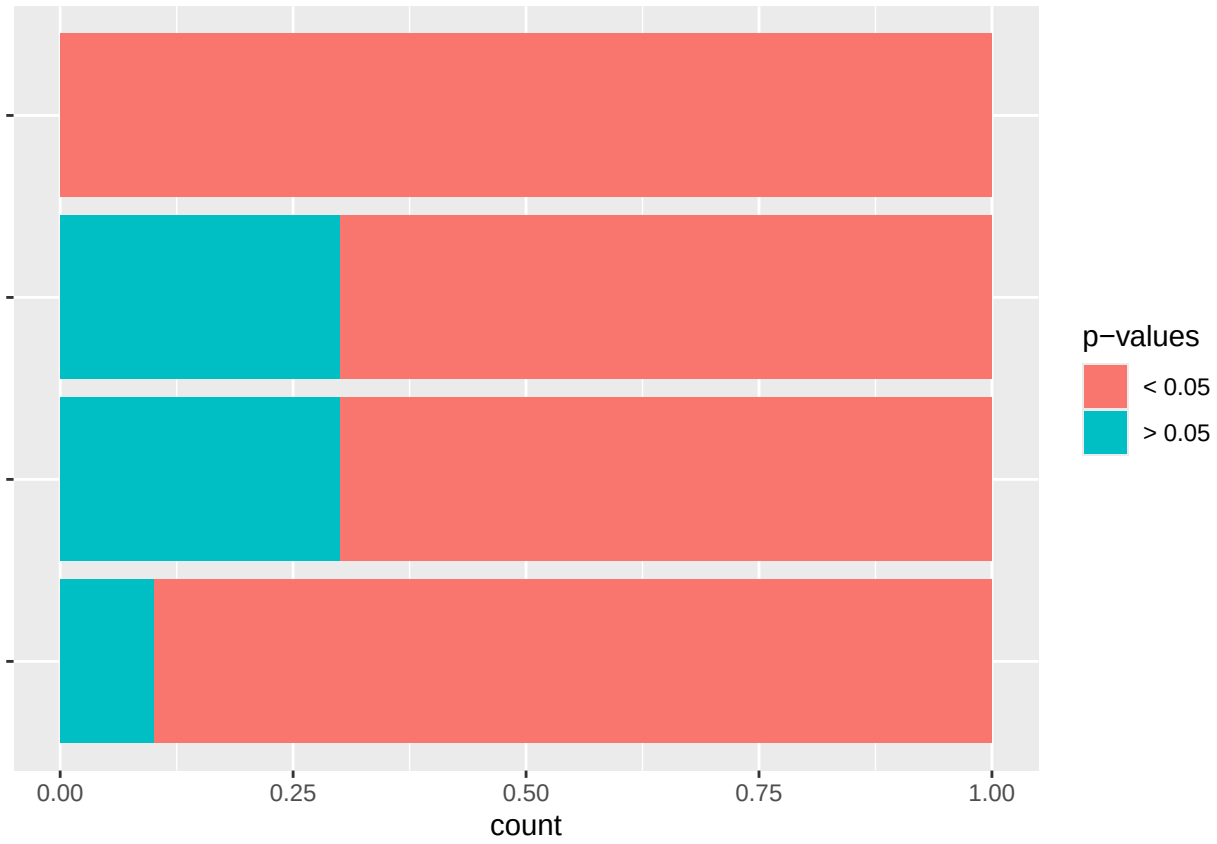
```
accuracy.plot <- ggplot(performance.knn.combined, aes(x = jitter(accuracy, amount = 0),
                                                      y = fct_reorder(comparison, accuracy),
                                                      fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                               quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = "Accuracy", option = "D") +
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +
  geom_point(shape = 124, size = 5, aes(x = baseline, y = comparison), color = "red") +
  xlab("Accuracy") +
  ylab("Comparison") +
  theme(legend.position = "none")
accuracy.plot
```

```
## Picking joint bandwidth of 0.00404
```

P-value Distributions

Distributions of p-values by comparison.

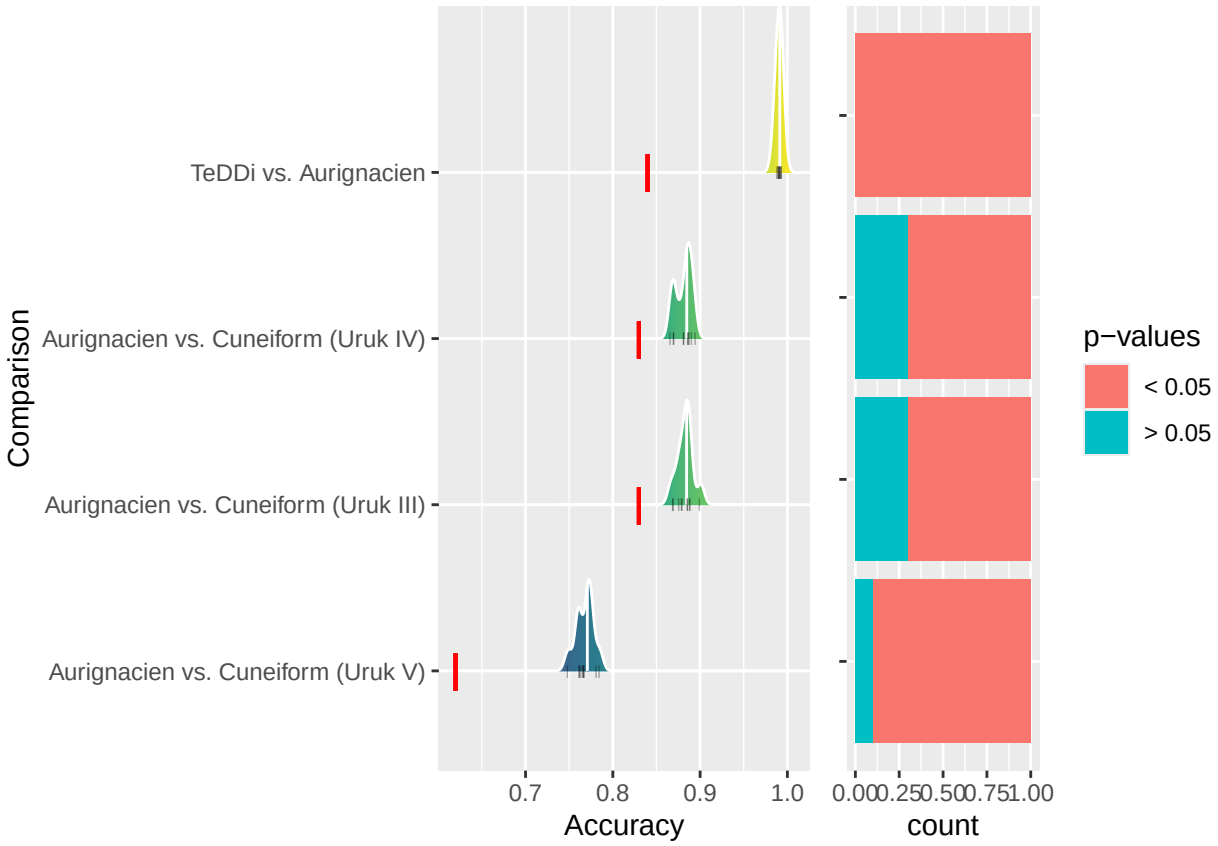
```
# get split between cases where p < 0.05 and p > 0.05
performance.knn.combined$accuracy.pvalue.corrected <-
  ifelse(performance.knn.combined$accuracy.pvalue.corrected < 0.05, "< 0.05", "> 0.05")

# create plot
pvalue.plot <- ggplot(performance.knn.combined, aes(y = fct_reorder(comparison, accuracy),
                                                    fill = as.factor(accuracy.pvalue.corrected))) +
  geom_bar(stat = "count", position = "fill") +
  theme(axis.title.y = element_blank(),
        axis.text.y = element_blank()) +
  scale_fill_discrete(name = "p-values")
pvalue.plot
```



Combine Plots

```
combined.plot <- grid.arrange(accuracy.plot, pvalue.plot, ncol = 2, widths = c(2, 1))
```

```
## Picking joint bandwidth of 0.00412
```



Safe to file.

```
ggsave("~/Github/UpperPalCombinatorics/figures/performancePlot_KNN.pdf",
       combined.plot, dpi = 300, width = 10, height = 2, device = cairo_pdf)
```